NeuroMIMO

THE PROJECT EXPLAINED

Abbreviations • Key Concepts • Architecture • Workflows • Results

A plain-English guide to the AI-enhanced 2×2 MIMO-OFDM channel estimation platform: what it does, how it is built, and how data flows through it.

October 25, 2026

Contents

1. What Is NeuroMIMO? 2
2. List of Abbreviations 3
3. Key Concepts 5
4. System Architecture 7
5. Workflows 9
6. Dashboard Pages and API 11
7. What the Results Show 12
8. Technology Stack 13
9. How to Run It 13

1. What Is NeuroMIMO?

NeuroMIMO is a software project that simulates how a modern wireless link (like 5G or Wi-Fi) sends data through the air, and compares three ways for the receiver to work out how the air has distorted the signal. One of those ways is an artificial-intelligence model (a deep neural network). Everything runs on a normal computer, with no radio hardware.

The Idea in Simple Words

Imagine shouting a message across a busy hall. Your voice bounces off walls, arrives as several echoes, and mixes with background noise. If the listener first hears a few *known* words, they can learn how the hall distorts sound and then understand the rest of your message. In wireless systems those known words are called **pilots**, and working out the distortion is called **channel estimation**.

NeuroMIMO compares three "listeners": **Least Squares (LS)**, the simplest; **Linear MMSE**, a smarter statistical method; and a **Deep Neural Network (DNN)** that learns from thousands of examples. It measures how accurate, how reliable and how fast each one is, and shows the results in an interactive web dashboard.

What the Project Does

- Simulates a 2×2 MIMO-OFDM link: 2 transmit and 2 receive antennas, 64 subcarriers, QPSK or 16-QAM, Rayleigh multipath fading or a plain noise (AWGN) channel.
- Estimates the channel with LS, LMMSE and a residual DNN, plus a "perfect knowledge" reference that shows the best possible result.
- Trains the DNN in PyTorch, from the command line or from the dashboard, with a live loss chart.
- Runs repeatable experiments (SNR sweep, AWGN vs Rayleigh, generalisation, pilot density, speed) and saves tables and 300 DPI figures.
- Can also use realistic ray-traced channels from DeepMIMO instead of the built-in channel model.
- Serves everything through a REST API (FastAPI) and a React dashboard with four colour themes.

2. List of Abbreviations

Every short form used in the project, the code and this document, in alphabetical order.

Abbreviation	Full form	Meaning in this project
3GPP	3rd Generation Partnership Project	The body that writes mobile standards such as 5G.
5G NR	Fifth Generation New Radio	The current mobile network standard.
Adam	Adaptive Moment Estimation	The optimiser used to train the neural network.
API	Application Programming Interface	The set of web addresses the frontend calls.
AWGN	Additive White Gaussian Noise	Random noise added to every received sample.
BER	Bit Error Rate	Fraction of received bits that are wrong.
BS	Base Station	The cell tower side of the link.
CFR	Channel Frequency Response	The channel gain on each subcarrier, written H[k].
CI	Confidence Interval / Continuous Integration	A range likely to hold the true value; or automatic testing on every push.
CIR	Channel Impulse Response	The channel in the time domain (one value per delay tap).
CLI	Command-Line Interface	A program you run from the terminal.
CNN	Convolutional Neural Network	A network that slides small filters over data.
CORS	Cross-Origin Resource Sharing	Browser rule for which websites may call the API.
CP	Cyclic Prefix	Copy of the end of each OFDM symbol placed at its start to absorb echoes.
CPU / GPU	Central / Graphics Processing Unit	Processors; the DNN runs on the CPU.
CSI	Channel State Information	Knowledge of the channel H at the receiver.
CSV	Comma-Separated Values	A simple spreadsheet file format.
CUDA	Compute Unified Device Architecture	NVIDIA's GPU computing platform (optional).
dB	Decibel	Logarithmic unit: 10·log10(ratio).
DFT	Discrete Fourier Transform	Converts between time and frequency; the FFT computes it fast.
DNN	Deep Neural Network	A neural network with several hidden layers.
DPI	Dots Per Inch	Image resolution; figures are saved at 300 DPI.
FFT / IFFT	(Inverse) Fast Fourier Transform	Fast algorithms for the DFT and its inverse.
GELU	Gaussian Error Linear Unit	The activation function used in the DNN.
HTTP	Hypertext Transfer Protocol	How the browser and backend talk.
ICI	Inter-Carrier Interference	Leakage between subcarriers (e.g. from Doppler).
IQ	In-phase and Quadrature	The real and imaginary parts of a complex symbol.
ISI	Inter-Symbol Interference	Overlap of one symbol's echoes into the next.
JSON	JavaScript Object Notation	Text format for data sent by the API.
KPI	Key Performance Indicator	The summary numbers on dashboard cards.
LMMSE / MMSE	(Linear) Minimum Mean Square Error	Estimator that uses channel statistics to remove noise.
LoS	Line of Sight	A direct path between transmitter and receiver.

Abbreviation	Full form	Meaning in this project
LS	Least Squares	Simplest estimator: received pilot divided by sent pilot.
MIMO	Multiple-Input Multiple-Output	Several antennas at both ends of the link.
MLP	Multilayer Perceptron	A fully connected neural network.
mmWave	Millimetre Wave	Very high radio frequencies (about 24–100 GHz).
MSE	Mean Squared Error	Average of squared errors; the DNN's training loss.
NMSE	Normalised Mean Squared Error	MSE divided by channel power; lower is better.
OFDM	Orthogonal Frequency-Division Multiplexing	Splits a wide band into many narrow subcarriers.
PDP	Power Delay Profile	How channel power is spread over delay taps.
QAM	Quadrature Amplitude Modulation	16-QAM sends 4 bits per symbol.
QPSK	Quadrature Phase Shift Keying	Sends 2 bits per symbol using 4 phases.
REST	Representational State Transfer	Style of web API using URLs and HTTP methods.
Rx / Tx	Receiver / Transmitter	The receiving and sending sides (or antennas).
SER	Symbol Error Rate	Fraction of received symbols that are wrong.
SISO	Single-Input Single-Output	One antenna at each end.
SNR	Signal-to-Noise Ratio	Signal power over noise power, in dB; higher is cleaner.
SQLite	Structured Query Language Lite	A small file-based database.
TTI	Transmission Time Interval	One scheduling slot in mobile systems.
UE	User Equipment	The phone or device side of the link.
UI	User Interface	The dashboard screens.
ZF	Zero-Forcing	Equaliser that inverts the channel matrix exactly.

3. Key Concepts

MIMO (multiple antennas)

With 2 antennas at each end, two data streams travel at the same time on the same frequency. The receiver separates them using its knowledge of the 2×2 channel matrix, so it must estimate four channel paths (each transmit antenna to each receive antenna).

OFDM (many narrow subcarriers)

The band is split into 64 subcarriers. Echoes would make a wideband signal hard to equalise, but each narrow subcarrier only sees one complex gain H[k]. A 16-sample cyclic prefix absorbs the echoes between OFDM symbols.

Pilots

Known symbols placed on fixed subcarriers. Transmit antenna 0 uses subcarriers 0, 16, 32, 48 and antenna 1 uses 8, 24, 40, 56, and each antenna stays silent on the other's pilots, so they never interfere. The remaining 56 subcarriers carry data.

The channel H

A Rayleigh channel has 4 delay taps with an exponential power delay profile: the direct, strongest path first and weaker echoes after it. In the frequency domain this gives a gain that rises and falls across the 64 subcarriers.

Noise and SNR

Random noise is added at a chosen signal-to-noise ratio. 0 dB means signal and noise are equally strong (very noisy); 30 dB means the signal is 1000 times stronger (very clean).

Least Squares (LS)

Divides each received pilot by the known pilot: $\hat{H}_{LS}[p] = Y[p] / X[p]$. It then fills in the other subcarriers with DFT interpolation. Fast and needs no prior knowledge, but it passes all the noise straight into the estimate.

Linear MMSE (LMMSE)

Combines the LS estimate with the known statistics of the channel and the noise level to suppress noise: $\hat{H} = \mathbf{R}_{HP}(\mathbf{R}_{PP} + \sigma^2\mathbf{I})^{-1}\hat{H}_{LS}$. Strong in theory, but it needs those statistics and a matrix inversion.

Residual DNN

A neural network that takes the LS estimate (512 numbers: real and imaginary parts of 2×2×64 values) and predicts a correction: $\hat{H}_{DNN} = \hat{H}_{LS} + f_\theta(\hat{H}_{LS})$. Layers $512 \to 256 \to 128 \to 256 \to 512$ with LayerNorm, GELU and dropout; 330,112 trainable parameters.

Equalisation

After estimating H, the receiver undoes the channel to recover the transmitted symbols. NeuroMIMO uses an MMSE equaliser below 15 dB SNR and zero-forcing at 15 dB and above.

Metrics

NMSE measures how close the estimated channel is to the true one (lower is better, in dB). BER and SER count wrong bits and symbols after the whole receiver. Every BER point has a 95% Wilson confidence interval, and zero-error points use the “rule of three” upper bound.

4. System Architecture

NeuroMIMO has three layers: a React frontend in the browser, a FastAPI backend that exposes the simulation as a web API, and a Python core that does the signal processing and machine learning. Results are stored in files and a small SQLite database.

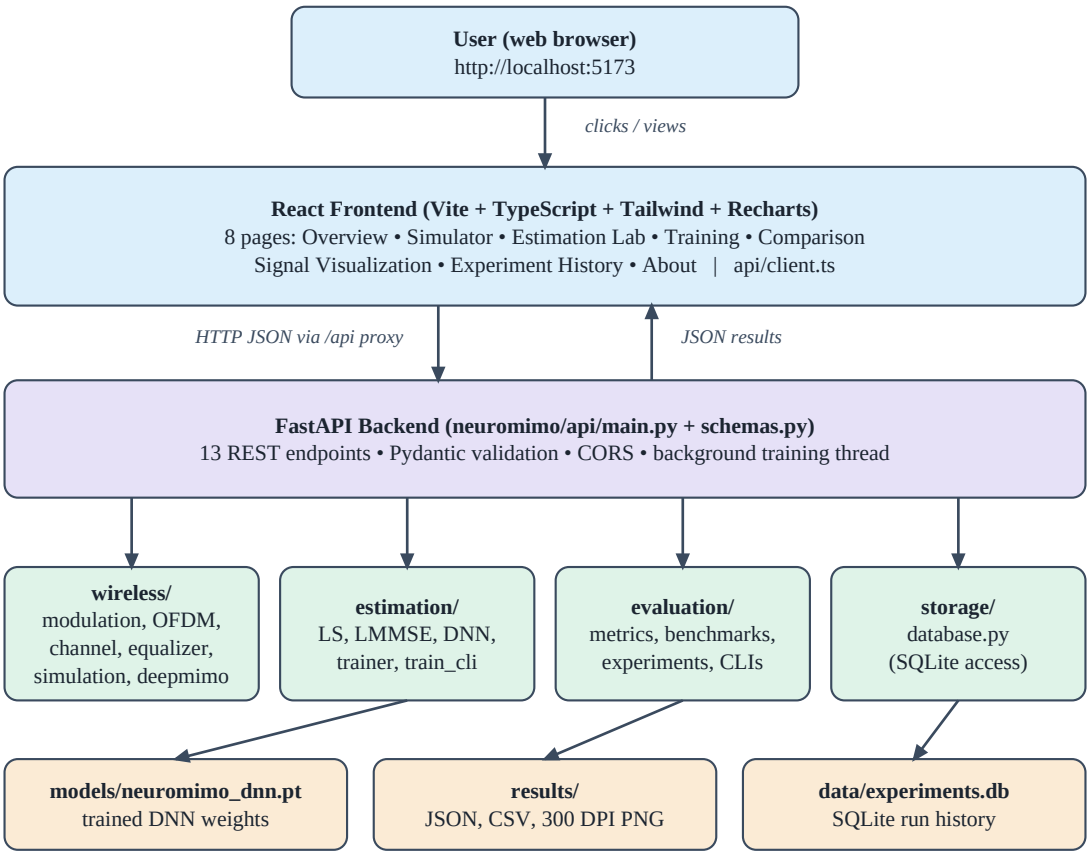


Figure 1: Overall system architecture.

Backend Modules

Package / file	Responsibility
`config.py`	Simulation settings (antennas, FFT size, pilots, SNR, seed) and enums.
`wireless/modulation.py`	Gray-coded QPSK and 16-QAM mapping and demapping; random bits.
`wireless/ofdm.py`	Pilot insertion, IFFT + cyclic prefix, CP removal + FFT, pilot extraction.
`wireless/channel.py`	AWGN and Rayleigh multipath channels; adds noise at the target SNR.
`wireless/deepmimo.py`	Loads DeepMIMO ray-traced channels and prepares them for the pipeline.
`wireless/equalization.py`	Zero-forcing and MMSE MIMO equalisers.
`wireless/simulation.py`	MIMOSimulationPipeline: runs one full transmission end to end.
`estimation/ls.py, mmse.py`	Least Squares (with DFT interpolation) and LMMSE estimators.
`estimation/dnn.py`	ChannelEstimationNet (residual MLP) and the DNNEstimator wrapper.
`estimation/trainer.py`	Dataset generation, leakage-free splitting, training loop, checkpoints.
`evaluation/metrics.py`	MSE, NMSE, BER, SER and confidence intervals.
`evaluation/experiments.py`	ExperimentRunner: SNR sweep, channel comparison, generalisation, pilot density, inference time; saves JSON/CSV/figures.
`storage/database.py`	SQLite tables for simulation runs and training runs.
`api/main.py, schemas.py`	FastAPI routes, request validation and the background training job.

Frontend Structure

Part	What it does
`src/api/client.ts`	Typed functions for every API call; shows backend error messages.
`src/theme.ts`	The four dark colour themes and saving the user's choice.
`components/layout/`	Navbar (API status, theme picker, refresh) and Sidebar (navigation).
`components/common/`	Reusable StatCard, Badge, Modal, LoadingSpinner, ErrorBanner.
`pages/`	One file per dashboard page (see Section 6).
`vite.config.ts`	Dev server on port 5173 that forwards /api calls to the backend on 8000.

5. Workflows

5.1 Signal Processing Pipeline (one transmission)

This is what `MIMOSimulationPipeline.run_single_shot()` does every time you press “Run Simulation”. One frame has 14 OFDM symbols; with QPSK that is 3,136 bits.

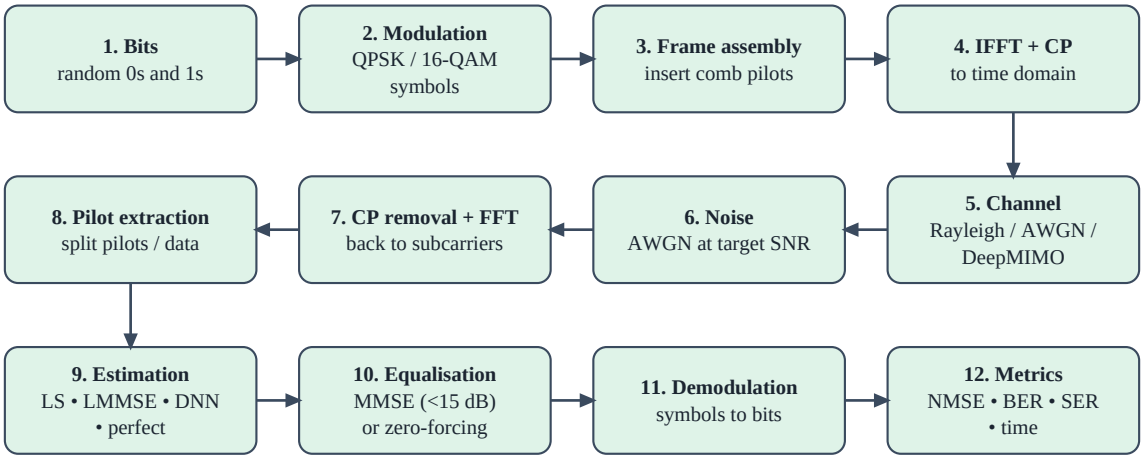


Figure 2: Transmit → channel → receive pipeline.

Step	What happens and why
1–2	Random bits are mapped to complex symbols (2 bits per QPSK symbol, Gray coded so a small error flips only one bit).
3	Each transmit antenna's 14×64 grid gets 4 pilots per symbol; the other antenna is silent on those subcarriers.
4	The IFFT turns subcarriers into a time signal; a 16-sample cyclic prefix guards against echoes.
5–6	The 2×2 channel H multiplies each subcarrier and noise is added so the received signal has exactly the requested SNR.
7–8	The receiver removes the prefix, applies the FFT and separates pilot and data subcarriers.
9	The chosen estimator produces $\hat{H}$ for all 64 subcarriers; NMSE compares it with the true H.
10–11	The equaliser undoes $\hat{H}$ on the data subcarriers and the demodulator recovers the bits.
12	Wrong bits and symbols are counted; constellation points and |H| are returned for the charts.

5.2 DNN Training Workflow

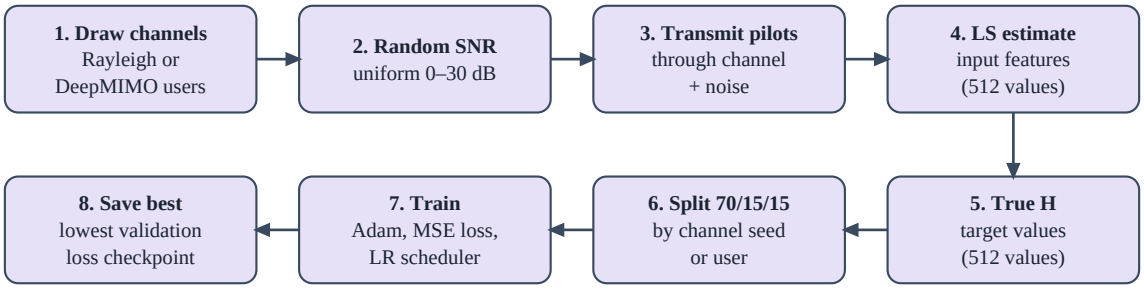


Figure 3: How the DNN learns (estimation/trainer.py).

Every synthetic sample uses its own freshly drawn channel, so splitting the samples 70/15/15 means the test set holds channels the network has never seen. With DeepMIMO the **users** are split first, so no user's channel appears in two sets. The input is only what a real receiver has (the noisy

LS estimate); the true channel is used only as the answer during training.

5.3 Dashboard Request Workflow

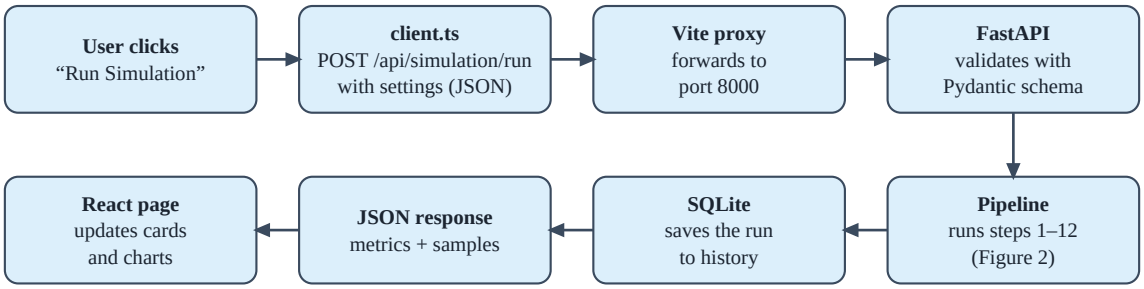


Figure 4: What happens between a click and the charts updating.

Training works the same way but runs in a background thread: the page starts a job with `POST /api/model/train` and then polls `GET /api/model/train/status` every second to draw the live loss curve.

5.4 Experiment Workflow

`python -m neuromimo.evaluation.run_experiments_cli` runs five experiments, and every estimator sees identical channels and noise (the random generator is reset with the same seed):

- **SNR sweep:** NMSE and BER at 0, 5, …, 30 dB.
- **Channel comparison:** flat AWGN versus Rayleigh multipath.
- **Generalisation:** different delay-profile decay values and a longer 6-tap channel.
- **Pilot density:** pilot spacing 4, 8 and 16.
- **Inference time:** 100 timed runs per estimator after 10 warm-up runs.

Results go to `backend/results/` as JSON, CSV and 300 DPI PNG figures, which the Performance Comparison page and these PDFs read.

5.5 DeepMIMO Workflow

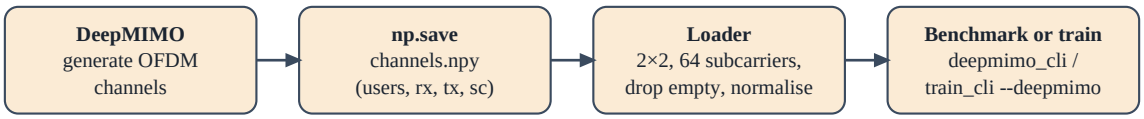


Figure 5: Using realistic ray-traced channels.

6. Dashboard Pages and API

Page	What you can do there
Overview Dashboard	Project summary, system status cards, pipeline steps and the research figures.
MIMO-OFDM Simulator	Choose antennas, modulation, channel, SNR, pilots and estimator; run one transmission.
Channel Estimation Lab	Run LS, LMMSE and DNN head to head on the same channel; compare true vs predicted H.
ML Model Training	Set epochs, batch size, learning rate and SNR range; train and watch the loss curve.
Performance Comparison	NMSE and BER waterfalls, latency, a metrics table and summary cards computed from data.
Signal Visualization	IQ constellation, channel frequency response, power delay profile and subcarrier grid.
Experiment History	Search and filter saved runs and export them to CSV.
About & Methodology	Theory, equations and how the evaluation was done.

The palette button in the top bar switches between the Neon, Nebula, Matrix and Solar dark themes.

REST API Endpoints

Method	Endpoint	Purpose
GET	`/api/health`	Backend status, device and library versions
GET	`/api/config`	Supported settings and defaults
POST	`/api/simulation/run`	Run one end-to-end transmission
POST	`/api/estimation/compare`	Compare several estimators on one channel
POST	`/api/model/train`	Train the DNN (now or in the background)
GET	`/api/model/status`	Is a trained model available, and how big is it
GET	`/api/model/train/status`	Live training progress
POST	`/api/model/predict`	Run the DNN on a channel
GET	`/api/experiments`	List saved runs
GET	`/api/experiments/{id}`	One saved run
GET	`/api/experiments/{id}/plots`	List result figures
GET	`/api/experiments/{id}/plots/{name}`	Download one figure
GET	`/api/experiments/benchmark/snr-sweep`	The pre-computed SNR sweep results

7. What the Results Show

Rayleigh channel, QPSK, 15 trials per SNR point. Numbers are read from `backend/results/`.

SNR	LS NMSE	LMMSE NMSE	DNN NMSE	LS BER	DNN BER
0 dB	−8.38 dB	−4.01 dB	−8.36 dB	0.2310 (10,866 err)	0.2318 (10,904 err)
10 dB	−18.24 dB	−14.71 dB	−17.15 dB	0.0642 (3,020 err)	0.0659 (3,102 err)
20 dB	−28.95 dB	−27.30 dB	−23.06 dB	0.0092 (434 err)	0.0091 (426 err)
30 dB	−38.64 dB	−38.41 dB	−24.76 dB	0.0018 (85 err)	0.0066 (311 err)

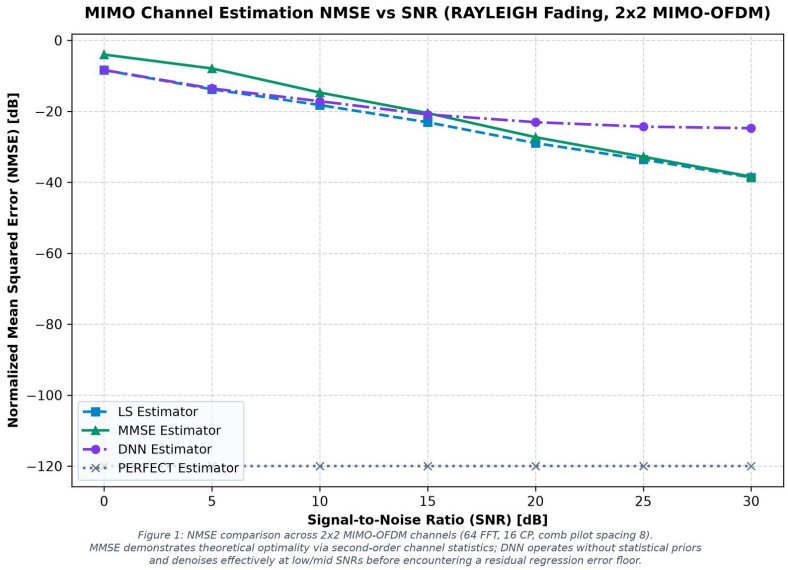


Figure 6: Channel estimation error versus SNR.

- **Noisy conditions (0–10 dB):** the DNN is as good as LS and clearly better than LMMSE at 0 dB (−8.36 dB vs −4.01 dB), without needing channel statistics.
- **Clean conditions (20–30 dB):** the DNN levels off near −24.76 dB while LS keeps improving to −38.64 dB. One small network trained on the whole 0–30 dB range cannot match LS when the noise is tiny.
- **Speed:** all estimators take under 1 ms per frame on a CPU (LS 0.12 ms, LMMSE 0.35 ms, DNN 0.49 ms).
- **Pilots matter:** with 2 pilots per antenna (spacing 16) every estimator fails, because a 4-tap channel needs at least 4 pilots to be resolved.

8. Technology Stack

Layer	Technology	Used for
Signal processing	Python, NumPy, SciPy	Modulation, FFT, channels, estimators, statistics
Machine learning	PyTorch	The residual DNN, training and inference
Figures	Matplotlib	300 DPI result figures
Backend	FastAPI, Uvicorn, Pydantic	REST API, validation, background training
Storage	SQLite	History of simulations and training runs
Frontend	React 19, TypeScript, Vite	The dashboard and its build tooling
Styling and charts	Tailwind CSS, Recharts, lucide-react	Themes, interactive charts, icons
Testing	pytest, FastAPI TestClient, oxlint	80 backend tests and frontend linting
Documents	ReportLab	The project PDFs in docs/

9. How to Run It

Backend (first terminal, from the project folder):

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r backend/requirements.txt
python -m pytest backend/tests -q
uvicorn neuromimo.api.main:app --app-dir backend --port 8000
```

Frontend (second terminal):

```
cd frontend
npm ci
npm run dev                        # then open http://localhost:5173
```

Useful commands (from backend/):

```
# Train the DNN
python -m neuromimo.estimation.train_cli --samples 500 --epochs 15
# Reproduce every experiment and figure
python -m neuromimo.evaluation.run_experiments_cli --trials 15
# Benchmark on DeepMIMO channels
python -m neuromimo.evaluation.deepmimo_cli channels.npy
```